16: Real-world Kubernetes security

In the previous chapter, you saw how to threat model Kubernetes using STRIDE. In this chapter, you'll cover some common security-related challenges that you're likely to encounter when implementing Kubernetes in the real world.

While every Kubernetes deployment is different, there are many similarities. As a result, the examples you'll see will apply to most Kubernetes deployments, large and small.

Now then, we're not offering *cookbook style* solutions. Instead, we'll be looking at things from the kind of high-level view a *security architect* has.

The chapter's divided into the following four sections:

- CI/CD pipeline
- Infrastructure and networking
- Identity and access management
- Security monitoring and auditing

CI/CD pipeline

Containers are a revolutionary application *packaging* and *runtime* technology.

On the packaging front, they bundle application code and dependencies into an *image*. As well as code and dependencies, images contain the commands required to run the application. This has enabled containers to hugely simplify the process of building, sharing, and running applications. It's also overcome the infamous "*it worked on my laptop*" issue.

However, containers make running dangerous code easier than ever before.

With this in mind, let's look at some ways you can secure the flow of application code from a developer's laptop to production servers.

Image Repositories

You store images in registries, and registries are either public or private.

> **Note**: Each registry is divided into one or more repositories, and you actually store images in repositories.

Public registries are on the internet and are the easiest way to download images and run containers. However, it's important to understand that they host a mixture of *official images* and *community images*. Official images are usually provided by product vendors and have undergone a vetting process to ensure certain levels of quality.

Typically, official images implement good practices, are regularly scanned for known vulnerabilities, contain up-to-date code, and may be supported by the product vendor. *Community images* are none of that. Yes, there are some excellent community images, but you should practice extreme caution when using them.

With all of this in mind, it's important you implement a standard way for developers to obtain and consume images. It's also vital that any such process be as frictionless as possible for developers – if there's too much friction, your developers will look for ways to bypass them.

Let's discuss a few things that might help.

Use approved base images

Images are made up of multiple layers that build on top of each other to form a useful image. But all images start with a base layer.

Figure 16.1 shows a simple example of an image with three layers. The base layer contains the core OS and filesystem components applications need in order to run. The middle layer contains the application library dependencies. The top layer contains the code that your developers have written. The combination of the three is an *image*, and it contains everything needed to run the application.

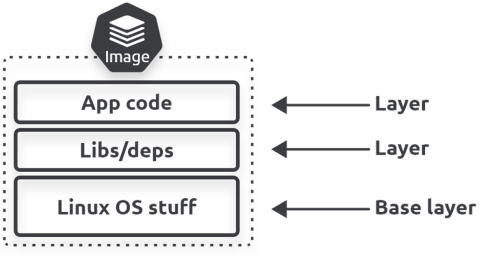

Figure 16.1

As all images have a base layer containing the required operating system (OS) and filesystem constructs for applications to build on, it's a common practice to have a small number of *approved base images*. It's also common, but not essential, for these base images to be derived from *official images*. For example, if you develop your applications on CentOS Linux, your base images *may* be based on the official CentOS image – you take the official CentOS base image and tweak it for your requirements.

In this model, all of your applications will build on top of a common approved base image like shown in Figure 16.2.

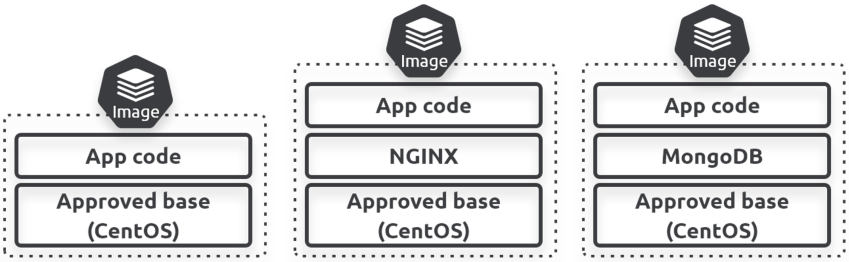

Figure 16.2

While there's up-front effort required to create and implement an approved set of base images, the long-term security benefits are worth it.

From a developer perspective. Developers can focus entirely applications and dependencies without worrying about maintaining OS stuff such as patching, drivers, audit settings, and more.

From an ops perspective. Base images reduce software sprawl. This makes testing easier, as you'll always be testing on a known base image. It makes pushing updates easier, you only need to update a small number of approved base images that can be easily made available to all developers. It also makes troubleshooting easier, as you have a small number of well-known base images providing your building blocks. It may also reduce the number of base image configurations that need tying into support contracts.

Non-standard base images

As good as it is to have a small number of approved base images, there may still be applications that need something more bespoke. This means you will need processes in place to

- Identify why an existing approved base image cannot be used
- Determine whether an existing approved base image can be updated to meet requirements (including if it's worth the effort)
- Determine the support implications of bringing an entirely new image into the environment

Generally speaking, updating an existing base image – such as adding a device driver for GPU computing – should be preferred over introducing an entirely new image.

Control access to images

There are several ways to protect your organization's container images. The most secure practical option is to host your own private registry within your own firewall. This allows you to manage how the registry is deployed, how it's replicated, and how it is patched. It may also enable you to integrate permissions with existing identity management providers, such as Active Directory, as well as allow you to create repositories that fit your organizational structure.

If you don't have the means for a dedicated private registry, you can host your images in private repositories on public registries such as Docker Hub. However, this is not as secure as hosting your own private registry within your own firewalled network.

Whichever solution you choose, you should only host images that are approved to be used within your organization. Normally, these will be from a *trusted* source and vetted by your information security team. You should place access controls on repositories that store these images, so that only approved users can push and pull them.

Away from the registry itself, you should also:

- Restrict which cluster nodes have internet access, keeping in mind that your image registry may be on the internet
- Configure access controls that only allow authorized users/nodes can `push` to repositories

Expanding on the list above…

If you're using a public registry, you'll probably need to grant your Nodes access to the internet so they can `pull` images. In this situation, a best practice is to limit internet access to the addresses and ports of any registries you use. You should also implement strong RBAC rules to maintain control over who is pushing and pulling images from which repositories. For example, developers should probably be able to `push` and `pull` from dev and test repositories, but not production. Whereas operations teams should probably be able to `pull` from non-production, as well as `push` and `pull` to production repos.

Finally, you may only want a sub-set of Nodes (*build nodes*) to be able to `push` images. You may even want to lock things down so that only your automated build systems can push to certain repositories.

Moving images from non-production to production

Many organizations have separate environments for development, testing, and production.

Generally speaking, development environments have less rules and are commonly used as places where developers can experiment. This can often involve non-standard images that your developers eventually want to use in production.

The following sections outline some measures you can take to ensure only safe images get approved into production.

Vulnerability scanning

Top of the list for vetting images before allowing them into production should be *vulnerability scanning*. This is a process where your images are scanned at a binary level and their contents checked against databases of known security vulnerabilities (CVEs).

If you have an automated CI/CD build pipeline, you should definitely integrate vulnerability scanning. As part of this, you should consider implementing policies that automatically fail builds and quarantine images containing certain categories of vulnerabilities. For example, you might implement a build phase that scans images and automatically fails anything using images with known *critical* vulnerabilities.

Two things to keep in mind if you do this…

Firstly, scanning engines are only as good as the vulnerability databases they use.

Secondly, scanning engines might not implement intelligence. For example, a method in Python that performs TLS verification might be vulnerable to Denial of Service attacks when the Common Name contains a lot of wildcards. However, if you never use Python in this way, the vulnerability *might not* be relevant and you *might* want to consider it a false positive. With this in mind, you may want to implement a solution that provides the ability to mark certain vulnerabilities as *not applicable*.

Configuration as code

Scanning app code for vulnerabilities is a widely accepted as good production hygiene. However, reviewing application configurations, such as Dockerfiles and Kubernetes YAML files, is less widely adopted.

The *build once, run anywhere* mantra of containers means a single container or Pod configuration can have hundreds or thousands of running instances. If any one of these configurations pulls in vulnerable code, you can easily end up running hundreds or thousands of instances of vulnerable code. With this in mind, if you are not already reviewing your Dockerfiles and Kubernetes YAML files for security issues, you should start now!

A well-publicised example of not reviewing configurations was when an IBM data science experiment embedded private TLS keys in its container images. This made it possible for an attacker to pull the image and gain root access to the Nodes that were hosting the containers. This wouldn't have happened if a security review had been performed against the application's Dockerfiles.

There continue to be advancements in automating these types of checks with tools that implement *policy as code* rules.

Sign container images

Trust is a big deal in today's world, and cryptographically signing content at every stage in the software delivery pipeline is becoming a *must have*. Fortunately, Kubernetes, and many container runtimes, support the ability to cryptographically sign and verify images.

In this model, developers cryptographically sign their images, and consumers cryptographically verify them when they `pull` and `run` them. This process gives the consumer confidence the image they're working with is the image they asked for and hasn't been tampered with.

Figure 16.3 shows the high-level image signing and verification process.

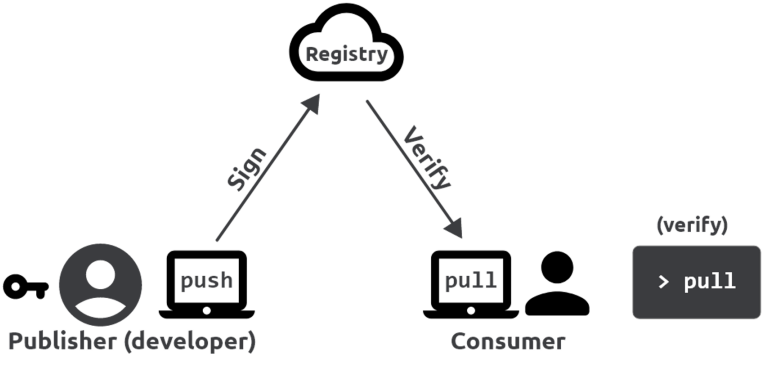

Figure 16.3

Image signing, and the verification of signatures, is usually implemented by the container runtime and Kubernetes doesn't get actively involved.

As well as signing images like this, higher-level tools often allow you to implement enterprise-wide policies that require certain teams to sign images before allowing them to be used.

Image promotion workflow

With everything that we've covered so far, a CI/CD pipeline for promoting an image to production should include as many of the following security-related steps as possible:

1. Configure environment to only `pull` and `run` signed images
2. Configure network rules to restrict which Nodes can `push` and `pull` images
3. Configure repositories with RBAC rules
4. Developers build images using approved base images
5. Developers sign images and push to approved repos
6. Images are scanned for known vulnerabilities
    - Policies dictate whether images are promoted or quarantined based on scan results
7. Security team:
    - Reviews source code and scan results
    - Updates vulnerability rating as appropriate
    - Reviews container and Pod configuration files
8. Security team signs the image
9. All image pull and container run operations verify image signatures

These steps are examples and not intended to represent an exact workflow.

Let's switch our focus away from images and CI/CD pipelines.

Infrastructure and networking

In this section, you'll a few ways you can isolate workloads.

We'll start at the cluster level, switch to the runtime level, and then look outside of the cluster at supporting infrastructure such as network firewalls.

Cluster-level workload isolation

Cutting straight to the chase, **Kubernetes doesn't support secure multi-tenant clusters**. **The only cluster-level security boundary in Kubernetes is the cluster itself**.

Let's look a bit closer…

The only way to divide a Kubernetes cluster is by creating *Namespaces*. A Kubernetes Namespace is not the same as a Linux kernel namespace, it's a logical partition of a single Kubernetes cluster. In fact, it's little more than a way of grouping resources and applying things like:

- Limits
- Quotas
- RBAC rules

The take-home point is that Kubernetes Namespaces do not guarantee Pod in one Namespace will not impact Pod in another Namespace. As a result, you should not run potentially hostile production workloads on the same physical cluster. The only way to run potentially hostile workloads, and guarantee true isolation, is to run them on separate clusters.

Despite this, Kubernetes Namespaces are useful and you *should* use them. Just don't use them as security boundaries.

Let's look at how Namespaces relate to *soft multi-tenancy* and *hard multi-tenancy*.

Namespaces and soft multi-tenancy

For our purposes, *soft multi-tenancy* is hosting multiple trusted workloads on shared infrastructure. By *trusted*, we mean workloads that don't require absolute guarantees that one Pod/container cannot impact another.

An example of trusted workloads might be an e-commerce application comprising a web front-end service and a back-end recommendation service. Both are part of the same e-commerce application, so are not hostile, but they might benefit from:

- Isolating the teams responsible for each service
- Having different resource limits and quotas for each service

In this situation, a single cluster with one Namespace for the front-end service and another for the back-end service might be a good solution. However, exploiting a vulnerability in one service might give the attacker access to Pods in the other service.

Namespaces and hard multi-tenancy

Let's define *hard multi-tenancy* as hosting untrusted and potentially hostile workloads on shared infrastructure. Only… as we said before, this isn't *currently* possible with Kubernetes.

This means truly hostile workloads – workloads that require a strong security boundary – need to run on separate Kubernetes clusters! Examples include.

- Isolating production and non-production workloads on dedicated clusters
- Isolating different customers on dedicated clusters
- Isolating sensitive projects and business functions on separate clusters

Other examples exist, but you get the picture. If you have workloads that require strong separation, put them on their own clusters.

> **Note:** The Kubernetes project has a dedicated *Multitenancy Working Group* that's actively working on the multitenancy models Kubernetes supports. This means that future releases of Kubernetes might support hard multitenancy.

Node isolation

There are times when individual applications require non-standard privileges, such as running as root or executing non-standard syscalls. Isolating these on their own clusters might be overkill, but the increased risk of collateral damage would probably justify running them on a ring-fenced subset of worker Nodes. In this case, if one Pod is compromised it can only impact other Pods on the same Node.

You should also apply *defence in depth* principles by enabling stricter audit logging and tighter runtime defence options on Nodes running workloads with non-standard privileges.

Kubernetes offers several technologies, such as labels, affinity and anti-affinity rules, and taints, to help target workloads to specific Nodes.

Runtime isolation

So far, we've looked at cluster-level isolation and Node-level isolation. Now let's look at the various types of runtime isolation.

Containers versus virtual machines can be a polarizing topic. However, when it comes to workload isolation there is only one winner… the virtual machine.

The most popular container model has multiple containers sharing a single kernel, with isolation provided by kernel constructs that were never designed as *strong* security boundaries. The technical term for these types of containers are called *namespaced containers*.

In the hypervisor model, every virtual machine gets its own dedicated kernel and is strongly isolated from other virtual machines using hardware enforcement.

From a workload isolation perspective, virtual machines win.

However, it's becoming easier and more common to augment containers with additional security technologies such as apparmor and SELinux, seccomp, capabilities, and user namespaces. Unfortunately, these can add significant complexity and may still be less secure than a virtual machine.

Another thing to consider, is different classes of container runtime. Two examples are **gVisor** and **Kata Containers**, both of which are re-writing the rules and providing stronger levels of workload isolation. Integrating runtimes like these with Kubernetes is made simple thanks to the Container Runtime Interface (CRI) and Runtime Classes.

There are also projects that enable Kubernetes to orchestrate other workloads such as virtual machines and serverless functions.

While all of this might feel overwhelming, everything discussed here needs to be considered when determining the levels of isolation your workloads require.

To summarize, the following workload isolation options exist:

1. **Virtual Machines**: Every workload gets its own virtual machine and kernel. It provides excellent isolation but is relatively slow and heavy-weight.
2. **Traditional namespaced containers**: Every workload gets its own container but shares a common kernel. Not the best isolation, but fast and light-weight.
3. **Run every container in its own virtual machine**: This option attempts to combine the versatility of containers with the security of VMs by running every container in its own dedicated VM. Despite using specialized lightweight VMs, this loses some of the appeal of containers and is not a popular solution.

4. **Use appropriate runtime classes:** This is a relatively new that allows you to run all workloads as containers, but target those requiring stronger isolation to an appropriate container runtime.

Finally, running a mix of containers and virtual machines can increase network complexity.

Network isolation

On the topic of networking, firewalls are an integral part of any layered security system. At a high level, they implement rules that *allow* or *deny* system-to-system communication.

As the names suggest, *allow rules* permit traffic to flow, whereas *deny rules* stop traffic flowing. The overall intent is to lock things down so only authorized communications occur.

In Kubernetes, Pods communicate with each other over a special internal network called the *Pod network*. However, Kubernetes does not implement the *Pod network*, instead, it implements a plugin model called the Container Network Interface (CNI). Vendors and the community are responsible for writing CNI plugins that actually provide the *Pod network*. There are lots of CNI plugins available, but they fall into two broad types:

- Overlay
- BGP

Each of these is different, and each has a different impact on firewall implementation and network security. Let's take a quick look at each.

Kubernetes and overlay networking

The most common way to build the *Pod network* is as an overlay network. In the Kubernetes world, overlay networking builds a simple flat Pod network that hides any complexity that might exist between the nodes in the cluster. For example, you might have your cluster deployed across two different subnets, but have all Pods on a single flat Pod network. In this scenario, the Pods only know about the flat overlay Pod network and have no knowledge of the networks the Nodes are on. Figure 16.4 shows four Nodes on two different networks, with Pods connected to a single overlay Pod network.

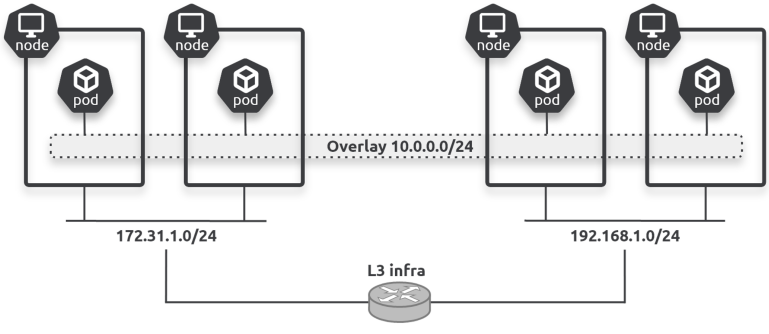

Figure 16.4

Generally speaking, overlay networks encapsulate packets for transmission over VXLAN tunnels. In this model, the overlay network is a virtual Layer 2 network operating on top of existing Layer 3 infrastructure. Traffic is

encapsulated in order to pass between Pods on different Nodes. This simplifies implementation, but encapsulation poses challenges for some firewalls. See Figure 16.5

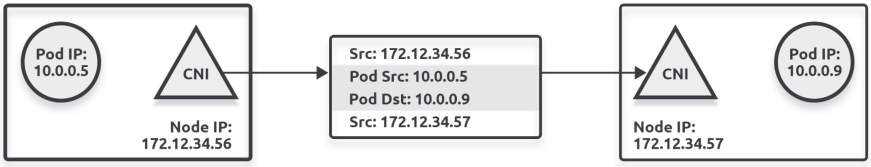

Figure 16.5

Kubernetes and BGP

BGP is the protocol that powers the internet. However, at its core it's a simple and scalable protocol that creates peer relationships that are used to share routes and perform routing.

The following analogy might help if you're new to BGP. Imagine you want to send a birthday card to a friend who you lost contact with and no longer have their address. However, your child has a friend at school whose parents are still in touch with your old friend. In this situation, you give the card to your child and ask them to give it to their friend at school. This friend gives it to their parents who deliver it to your friend.

This is similar to BGP. BGP Routing happens through a network of *peers* that help each other find a route for packets to go from one Pod to another.

BGP doesn't encapsulate packets, making life easier for firewalls. See Figure 16.6.

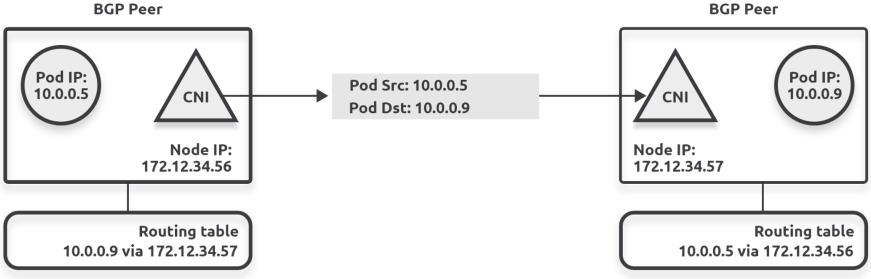

Figure 16.6

How this impacts firewalls

We've already defined a firewall as a network entity that allows or disallows traffic-flow based on source and destination addresses. For example:

- Allow traffic from the 10.0.0.0/24 network
- Disallow traffic from the 192.168.0.0/24 network

If your Pod network is an overlay network, source and destination Pod IP addresses are encapsulated so they can traverse the underlay network. This means only firewalls that crack open packets and inspect the contents

will be able to filter on Pod source and Pod destination IPs. You should consider this when choosing your Pod network and firewall solutions.

With this in mind, if your Pod-to-Pod traffic has to traverse existing firewalls that don't perform deep packet inspection, it might be a better idea to choose a BGP Pod network. This is because BGP doesn't obscure Pod source and destination addresses.

You should also consider whether to deploy *physical firewalls*, *host-based firewalls*, or a combination of both.

Physical firewalls are dedicated network hardware devices that are usually managed by a central team. Host-based firewalls are operating systems (OS) features and are usually managed by the team that deploys and manages your OSes.

Both solutions have their pros and cons, and a combination of the two is often the most secure. However, you should consider things such as whether your organization has a long and complex procedure for implementing changes to physical firewalls. If it does, it might not suit the nature of your Kubernetes deployment, and a different firewall solution might be preferable.

Packet capture

On the topic of networking and IP addresses, not only are Pod/container IP addresses sometimes obscured by encapsulation, they are also dynamic and can be recycled and re-used by different Pods and containers. This causes a lot of *IP churn* and reduces how useful IP addresses are in identifying systems and workloads. With this in mind, the ability to associate IP addresses with Kubernetes-specific identifiers such as Pod IDs, Service aliases, and container IDs when performing things like packet capturing is extremely useful.

Let's switch tack and look at some ways of controlling user access to Kubernetes.

Identity and access management (IAM)

Controlling user access to Kubernetes is important in any production environment. Fortunately, Kubernetes has a robust RBAC subsystem that integrates with existing IAM providers such as Active Directory and other LDAP systems.

Most organizations already have a centralized IAM provider that's integrated with company HR systems to simplify employee lifecycle management.

Fortunately, Kubernetes leverages existing IAM providers instead of implementing its own. For example, a new employee joining the company will automatically get an identity in Active Directory, which integrates with Kubernetes RBAC and can grant that user certain permissions in Kubernetes. Likewise, an employee leaving the company will automatically have his or her Active Directory identity removed or disabled, resulting in their access to Kubernetes being revoked.

RBAC went GA in Kubernetes 1.8 and it is highly recommended that you leverage its full capabilities. See the chapter on RBAC and API security for more info.

Managing Remote SSH access to cluster nodes

Almost all Kubernetes administration is done via the API server, meaning it should be rare for a user to require remote SSH access to Kubernetes cluster nodes. In fact, remote SSH access to cluster nodes should only be for the following types of activity.

- *Node management* activities that cannot be performed via the Kubernetes API
- *Break the Glass* activities such as when the API server is down
- Deep troubleshooting

You should probably have tighter controls over who has remote access to control plane nodes.

Multi-factor authentication (MFA)

With great power comes great responsibility…

Accounts with administrator access to the API server, and root access to cluster nodes, are extremely powerful and are prime targets for attackers and disgruntled employees. As such, their use should be protected by multi-factor authentication (MFA) where possible. This is where a user has to input a username and password followed by a second stage of authentication. For example.

- Stage 1: Tests *knowledge* of a username and password
- Stage 2: Tests *possession* of something like a one-time password device/app

You should also secure access to workstations and user profiles that have kubectl installed.

Auditing and security monitoring

No system is 100% secure, and you should plan for the eventuality that you'll be breached. When breaches happen, it is vital you can do at least two things:

1. Recognize that a breach has occurred
2. Build a detailed timeline of events that cannot be repudiated

Auditing is key to both of these, and the ability to build a reliable timeline helps answer the following post-event questions; *what happened, how did it happen, when did it happen* and *who did it…* In extreme circumstances, information like this can even be called upon in court.

Good auditing and monitoring solutions also help to identify vulnerabilities in your security systems.

With these points in mind, you should ensure reliable auditing and monitoring is high on your list of priorities, and you shouldn't go live in production without them.

Secure Configuration

There are various tools and checks that can be useful in ensuring your Kubernetes environment is provisioned according to best practices and in-line with company policies.

The Center for Information Security (CIS) has published an industry standard benchmark for Kubernetes security, and Aqua Security (aquasec.com) has written an easy-to-use tool called `kube-bench` to implement the CIS tests. In its most basic form, you run `kube-bench` against each node in your cluster and get a report outlining which tests passed and which failed.

Many organizations consider it a best practice to run `kube-bench` on all production nodes as part of the node provisioning process. Then, depending on your risk appetite, you can pass or fail provisioning tasks based on the results.

`kube-bench` reports can also serve as a valuable baseline in the aftermath of an incident. In situations like this, you run an additional `kube-bench` after a breach and compare the results with the initial baseline to determine *if* and *where* the configuration has changed.

Container and Pod lifecycle Events

As previously mentioned, Pods and containers are ephemeral in nature, meaning they don't live for long – certainly not as long as VMs and physical servers. This means you'll see a lot of events announcing new Pods and containers, as well as a lot of events announcing terminated Pods and containers. It also means you may need a solution that stores logs in an external system and keeps them around for a while after their Pods and containers have terminated. If you don't, you *may* find it frustrating that you don't have logs for old terminated containers available for inspection.

Logs entries relating to container lifecycle events may also be available from your container runtime (engine) logs.

Application logs

In some situations, there's not a lot Kubernetes can do to protect the applications it runs. For example, Kubernetes can't prevent an application from running vulnerable code. This means it is important to capture and analyse application logs as a way to identify potential security-related issues.

Fortunately, most containerized applications send log messages to `standard out (stdout)` and `standard error (stderr)`. These are then directed to the container's logs. However, some applications send log messages to other locations such as proprietary log files, so be sure to check your application's documentation.

Actions performed by users

Most of your Kubernetes configuration will be done via the API server where all requests should be logged. However, it's also possible to gain remote SSH access to control plane nodes and directly manipulate Kubernetes objects. This may include local unauthenticated access to the API, as well as directly modifying control plane systems such as etcd.

We've already spoken about limiting who has remote SSH access to Masters and Nodes, and bolstering security via things like multi-factor authentication. However, logging all activities performed via SSH sessions, and shipping them to a secure log aggregator is highly recommended. As is the practice of always having a second pair of eyes involved in remote access sessions.

Managing log data

A key advantage of containers is application density – you can run a lot more applications on your servers and in our datacenters. While this is great, it has the side-effect of generating massive amounts of logging and audit data that can easily become too much to analyse with traditional tools. At the time of writing, there is a lot of work being done to resolve this, including areas such as machine learning, but there is currently no easy solution.

On the negative side, such vast amounts of log-related data makes proactive analysis difficult – too much data to analyse. However, on the positive side, you have a lot of valuable data that can be used by security first-responders, as well as for post-event reactive analysis.

Migrating existing apps to Kubernetes

Every business has a mix of apps – some more business critical than others. With this in mind, it's important to adopt a careful and planned approach to migrating apps to Kubernetes.

One approach may be a crawl, walk then run strategy as follows:

1. *Crawl*: Threat modeling your existing apps will help you understand the current security posture of those applications. For example, which of your existing apps do and don't communicate over TLS.
2. *Walk*: When moving to Kubernetes, ensure the security posture of these apps remains unchanged; neither lower nor higher, just the same. For example, if an app doesn't communicate over TLS, do **not** change this as part of the migration.
3. *Run*: Start improving the security of applications after the migration is successful. Start with the simple non-critical apps, and carefully work your way up to the mission critical ones. You may also want to methodically deploy deeper levels of security. For example, initially configure apps to communicate over one-way TLS and then eventually over two-way TLS (potentially using a service mesh).

Real world example

A great example of a container-related vulnerability, that can be prevented by implementing some of the best practices we've discussed, occurred in February 2019. CVE-2019-5736 allowed a container process running as `root` to escape its container and gain root access on the host **and** all containers running on the host.

As dangerous as the vulnerability is, the following things that we covered in this chapter would've prevented the issue.

- Vulnerability scanning
- Not running processes as root
- Enabling SELinux

As the vulnerability has a CVE number, security scanning tools would've found it and alerted on it. Also, organizations that don't allow container processes to run as root will have been protected. Finally, common SELinux policies, such as those that ship with RHEL and CentOS, prevented the issue.

All in all, a great real-world example of the benefits of defence-in-depth and other security-related best practices.

Chapter summary

The purpose of this chapter was to give you an idea of some of the real-world security considerations effecting many Kubernetes clusters.

We started out by looking at ways to secure the software delivery pipeline by discussing some image-related best practices. These included how to secure your image registries, scanning images for vulnerabilities, and cryptographically signing and verifying images. Then we looked at some of the workload isolation options that exist at different layers of the infrastructure stack. In particular, we looked at cluster-level isolation, node-level isolation, and some of the different runtime isolation options. We talked about identity and access management, including places where additional security measures might be useful. We then talked about auditing, and finished up with a real-world issue that could be easily avoided by implementing some of the best practices already covered.

Hopefully you now have enough to go away and start securing your own Kubernetes clusters.